

Multi-Sport Match Outcome Predictor

Full-stack ML platform forecasting match outcomes across five sports

Role: ML systems builder

Python

Flask

scikit-learn

pandas

SQLite

Docker

Developed a full-stack ML platform that predicts match outcomes across five sports using engineered features, adaptive retraining, injury adjustments, and confidence scoring, served through Flask REST APIs.

66.8%

Model Accuracy

Across 1,140+ Premier League matches

+17%

Lift vs Baseline

Outperformed a naive predictor

22%

Faster Feature Extraction

SQL improved through profiling

Problem

Sports outcomes are noisy, so meaningful lift over a baseline is hard to get. Raw win probabilities are also useless if they're poorly calibrated or unexplained.

Solution

Feature pipelines and model-selection loops benchmark against a baseline, track calibration, and explain each prediction. A dashboard adds history, injury adjustments, and what-if scenarios.

Architecture

- Flask app factory with page, REST API, and health/readiness routes
- Service layer for predictions, history, and injuries
- Adaptive per-sport training with versioned, rollback-safe model artifacts
- Premier League pipeline with rolling and exponentially weighted form features
- SQLite persistence for history, injuries, and model runs
- Docker Compose and GitHub Actions with pytest and ruff

Highlights

- 66.8% accuracy across 1,140+ Premier League matches, 17% above a naive baseline
- Covers football, basketball, American football, volleyball, and cricket
- Adaptive per-sport retraining with time-series cross-validation and calibration tracking
- Injury adjustments, confidence tiers, and top-factor explanations for each prediction
- Flask REST APIs, SQLite prediction history, and what-if simulations
- SQL feature extraction made 22% faster through profiling

Challenges and Tradeoffs

- Challenge: High variance in match outcomes. Change: Expanded features (xG, points, and rest differentials) and benchmarked against a baseline continuously.
- Challenge: Feature extraction was slow on larger sets. Change: Profiled and rewrote the SQL, 22% faster.

- Challenge: Live sports feeds are flaky. Change: Automatic fallback to deterministic demo data, with a banner in the UI.
- Tradeoff: Model complexity vs interpretability. Decision: Kept the pipeline explainable while still improving lift.

Outcomes

- Measurable improvement in a noisy prediction domain
- A model-health endpoint reports status (Healthy, Needs More Data, Undertrained)
- Reusable feature-engineering templates for future experiments

Repository: <https://github.com/Jaskeeratr/Premier-predictor> · Full case study: <https://jaskeerat-portfolio.vercel.app/projects/premier-league-predictor>